

体系结构篇期末复习

依据：课程 PPT、123.txt 答疑转写、语音版重点、考试题目类型与重点；复习提纲只作范围参考。

使用建议

体系结构题通常不是背定义就结束，而是让你基于同一个系统画物理包图、说明分层/MVC、设计展示层到逻辑层接口、逻辑层到数据层接口，并解释 PO/VO、接口和依赖方向。复习时请把这份材料当成答题模板使用。

一、考试定位与优先级

优先级	内容	为什么重要	答题产物
高	分层体系结构、MVC、物理包图	答疑和往年卷都反复出现，老师明确说要会画	浏览器/服务器、展示层/逻辑层/数据层、REST API、模块包
高	接口设计、PO/VO、DAO/Repository	往年结构题第2问常让写接口声明和包名	Service 接口、Data/DAO 接口、VO/PO 参数与返回值
中高	体系结构定义、三要素、4+1 视图	适合简答题，也能支撑包图解释	部件、连接件、配置；逻辑/开发/进程/物理/场景视图
中	风格选择与质量属性	课件强调需求会影响架构风格	根据并发、安全、可维护性、可移植性选择风格
低到中	包的原则	今年答疑说包的原则不是重点，但包划分思想仍有帮助	只需知道职责单一、依赖方向清楚、避免循环依赖

一句话判断

这部分最像一道工程设计题：先用分层/MVC定系统骨架，再用包图把模块放进层里，最后用接口把层间交互说清楚。

二、体系结构基础概念

1. 什么是软件体系结构

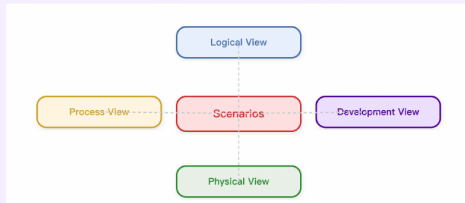
软件体系结构是系统的高层设计，描述系统由哪些大的部件组成、部件之间如何连接、以及这些部件如何被组织和部署。课程中可以记为：软件体系结构 = 部件 Component + 连接件 Connector + 配置 Configuration。

- 部件：承担功能或数据职责的基本单位，例如包、模块、服务、数据访问层、数据库。
- 连接件：部件之间交互的方式，例如方法调用、REST API、HTTP、数据库连接、事件消息。
- 配置：部件和连接件组成的整体拓扑，例如三层架构、客户端/服务器部署、MVC 组织方式。
- 为什么重要：体系结构是最难改的设计决策，影响可维护性、可测试性、性能、安全、团队协作和后续演化。

2. 体系结构设计和详细设计的区别

维度	体系结构设计	详细设计
粒度	系统、层、包、模块、服务	类、方法、对象协作、算法
核心问题	系统有哪些大模块？模块怎么连接？部署在哪里？	每个模块内部由哪些类实现？对象之间如何协作？
常见图	包图、组件图、部署图、4+1 视图	类图、详细顺序图、状态图、流程图
考试联系	给出分层包图和接口	根据接口和用例继续设计类、顺序图、设计原则

Kruchten 4+1 视图模型



每个视图面向不同的利益相关者，解决不同的关注点。

27

课件截图：4+1 视图模型。考试中重点理解逻辑视图常用包图表达，场景视图用用例验证架构。

三、体系结构风格

1. 风格的含义

体系结构风格是一组高层设计决策，它规定系统中有哪些典型部件、部件之间用什么连接件交互，以及这些连接和组织需要遵守什么约束。它和设计模式的区别是：风格解决系统整体组织问题，模式通常解决局部设计问题。

风格	核心结构	适用场景	答题要点
主程序-子程序	主控模块调用多个子程序	功能流程清晰、控制集中	简单直接，但主控容易膨胀，复用和演化较弱
面向对象	对象封装数据和行为，通过消息协作	领域对象清晰、需要复用和扩展	封装、继承、多态，便于变化隔离
分层	展示层、逻辑层、数据层等，上层依赖下层接口	Web/管理信息系统，课程和考试重点	层间接口清楚，降低耦合，便于替换和测试
MVC	Model、View、Controller 分离	Web 应用、GUI、交互逻辑复杂系统	把界面显示、用户输入控制、业务数据模型分开
客户端/服务器	客户端发起请求，服务器提供服务	浏览器/服务器、移动端/后端	要体现跨网络、HTTP、REST API、服务器部署

步骤 2：选择体系结构风格

需求特征	推荐风格	原因
企业级业务系统、团队分工	分层	职责分明、易于分工
数据流式处理、转换链	管道-过滤器	数据逐步变换
高交互、多视图同步	MVC	界面与逻辑分离
大规模、独立部署、多团队	微服务	独立开发/部署/扩展
松耦合、异步通信	事件驱动	发布/订阅解耦
小型工具、脚本	主程序-子程序	简单直接

实践建议：大多数企业系统首选分层架构，在此基础上根据需要引入其他风格。

40

课件截图：体系结构风格选择。非功能需求会影响风格，例如高并发、安全性、可维护性。

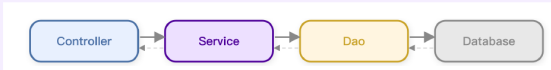
2. 分层风格

- 典型三层：展示层 Presentation 负责界面和交互，逻辑层 Business/Service 负责业务规则和用例处理，数据层 Data/DAO/Repository 负责持久化访问。
- 严格分层：只允许调用相邻下层，依赖方向清楚；松散分层：为了效率可跨层调用，但会增加耦合，考试答题优先画严格分层。
- 层之间不要直接依赖具体实现，尽量依赖接口。这样数据层实现可以替换，逻辑层也更容易测试。
- Web 题里常把浏览器放客户端，后端逻辑层和数据层放服务器端，通过 HTTP/REST API 连接。

案例：在线书店系统分层架构

层次	包名	关键类
展示层	presentation	BookController, OrderController, UserController
业务逻辑层	service	BookService, OrderService, UserService
数据访问层	dao	BookDao, OrderDao, UserDao
实体层	entity	Book, Order, User, CartItem

调用链路



21

课件截图：分层架构案例。复习时注意每一层内部还可以按业务模块横向拆包。

3. MVC 和分层的关系

MVC 不是简单等同于三层架构。MVC 主要解决界面交互组织问题，强调 View、Controller、Model 的职责分离；分层架构解决系统整体职责和依赖问题。Web 系统中常把 View 放展示层，Controller 接收请求并调用 Service，Model/领域对象和业务逻辑在逻辑层，持久化放数据层。

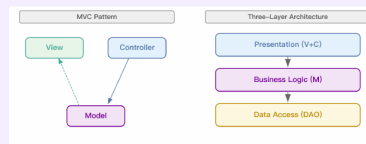
MVC 与分层风格的关系

映射关系

MVC	三层架构
View	展示层
Controller	展示层
Model	业务逻辑层 + 数据访问层

关键区别

- 分层关注纵向职责划分
- MVC关注交互逻辑分离
- 两者可以结合使用



实践中常将 MVC 嵌入展示层

25

课件截图：MVC 与分层之间的关系。考试画图时要把 MVC 放进层次结构里理解。

四、体系结构设计过程

- 1 识别关键需求：从业务目标、用例、非功能需求中找出会影响架构的需求，例如并发、安全、可维护性、跨网络部署、外部接口。
- 2 选择体系结构风格：根据关键需求选择分层、MVC、客户端/服务器等风格，并说明选择理由。
- 3 划分层和模块：先纵向分层，再横向按业务模块拆包，例如 user、order、inventory、recharge。
- 4 设计层间接口：展示层调用逻辑层接口，逻辑层调用数据层接口。接口方法来自用例流程和数据访问需要。
- 5 确定数据对象：展示层到逻辑层使用 VO，逻辑层到数据层使用 PO，避免把数据库细节暴露给界面。
- 6 物理部署和验证：说明客户端/服务器/数据库部署位置，通过场景、评审、集成测试验证架构能支撑需求。

体系结构设计过程 — 完整版

步骤	名称	核心任务
1	分析关键需求	识别功能需求、质量属性、项目约束
2	选择体系结构风格	根据需求特征选择合适风格
3	逻辑设计	将需求分配到子系统和模块
4	接口设计	定义模块间的交互接口
5	物理设计	决定部署方案（硬件/网络拓扑）
6	验证设计方案	检查是否满足需求和约束
7	评审与改进	团队评审，迭代优化

关键提示：步骤 3（逻辑设计）是最核心的步骤，需要反复迭代直到模块划分合理。

33

课件截图：体系结构设计过程。按这个顺序答题，比直接堆包名更稳。

架构显著需求 ASR

Architecturally Significant Requirement 指会明显影响体系结构选择的需求。例：必须通过浏览器访问，会推动 B/S 架构；必须支持高并发，会影响是否采用缓存、异步、微服务；必须保护隐私，会增加认证授权和数据访问边界。

五、4+1 视图与包图

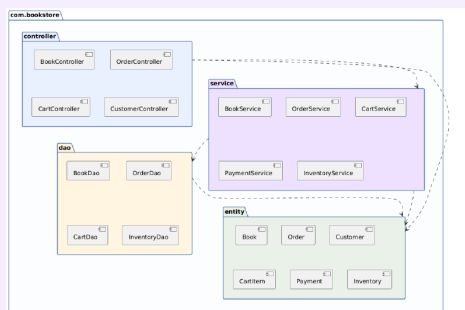
1. 4+1 视图怎么记

视图	回答的问题	常用图/内容
逻辑视图	功能如何被组织为类、包、模块？	包图、类图。考试画分层包图主要属于这里
开发视图	代码怎么组织、构建、分工？	包、Maven/Gradle 模块、目录结构
进程视图	运行时并发、通信、进程线程怎么协作？	进程/线程、消息、运行时交互
物理视图	系统部署在哪些机器和节点上？	部署图、客户端/服务器/数据库
场景视图	用哪些用例验证架构能工作？	关键用例、系统顺序图、端到端场景

2. 包图的组成部分

- 包 Package：表示一组职责相关的类或接口。体系结构包图中，一个包常代表一个模块或一层中的业务子模块。
- 层 Layer：展示层、逻辑层、数据层等，可以用大包或分区表达。
- 接口 Interface：层间交互的契约。展示层依赖逻辑层接口，逻辑层依赖数据层接口。
- 依赖 Dependency：通常用虚线箭头表示，一个包使用另一个包提供的能力。要注意依赖方向，不要让数据层反向依赖展示层。
- 实现 Realization：实现接口的关系，UML 中常用实线加空心三角形指向接口。
- PO/VO：放在包图中能说明数据跨层传递边界。VO 面向展示，PO 面向持久化。

案例：在线书店 — 逻辑视图（包图）



每个顶层包内部按业务模块再次划分（book / order / cart / customer）。

29

课件截图：在线书店逻辑视图包图。注意先分层，再按功能模块拆分包。

六、包图绘制答题模板

1. 拿到实际系统时怎么画

- 1 先看部署要求：题目若说 Web、浏览器、服务器、HTTP、REST，就必须画出客户端和服务端，中间标 HTTP/REST API。
- 2 确定层：客户端一般有 HTML/CSS/JS 或 View/Controller；服务器端至少有逻辑层 Service 和数据层 DAO/Repository，必要时画数据库。
- 3 按用例找功能模块：从题目中的业务功能抽包，例如库存、销售订单、采购询价、充值、账户、明细、绑定银行卡。
- 4 给每层都放对应模块：例如 recharge.view/recharge.service/recharge.dao，而不是只画一个“充值”大包。
- 5 补接口和数据对象：展示层到逻辑层画 Service 接口和 VO；逻辑层到数据层画 Data/DAO 接口和 PO。
- 6 标清依赖方向：上层依赖下层接口；具体实现依赖接口；避免 dao 依赖 service、entity 依赖 controller 这类违规。

图中元素	考试应体现什么
客户端	browser/client，包含 html、css、js、view、controller 或 API client
网络连接	HTTP，REST API。若题目要求跨网络，这个必须画
服务器端展示/API层	Controller/Resource，可接收 REST 请求，调用业务服务
逻辑层	Service 接口、ServiceImpl、业务模块。方法根据用例来定
数据层	DAO/Repository/DataService 接口和实现，负责查询和保存 PO
对象包	VO 用于展示层和逻辑层之间，PO 用于逻辑层和数据层之间
数据库	DB/table，可作为数据层外部持久化节点

2. 常见扣分点

- 只画功能模块，不画层次。体系结构题要体现结构，不是功能清单。
- 没有体现客户端/服务器和 HTTP/REST，尤其是往年卷明确要求 Web 框架时。
- 接口和实现混在一起，或没有写包名。接口题要写 interface 声明，并说明所属包。
- VO/PO 用反：VO 面向展示层，PO 面向数据库持久化。
- 依赖方向反了，例如数据层调用展示层，或底层依赖高层具体实现。
- 把 Spring 注解、配置细节当重点。答疑明确说具体技术细节不是考试重点。

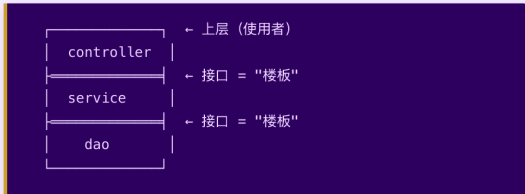
七、接口、PO/VO 与 DAO

1. 接口在体系结构中的作用

体系结构中的接口比 Java 语法中的 interface 更宽。它不仅表示某个类实现的接口，也表示模块之间交互的契约：一个模块向外提供什么能力，同时它需要调用别的模块什么能力。完整描述一个构件时，要同时考虑供接口 Provided Interface 和需接口 Required Interface。

接口在体系结构中的作用

"楼板"比喻



接口是层与层之间的**契约**，如同楼板将楼层隔离。

接口的三大好处

1. **解耦**：上层不依赖下层实现
2. **可替换**：实现类可以随时替换
3. **可测试**：用 Mock 替代真实实现

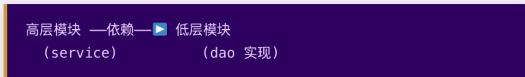
先定义接口，再编写实现 — 这就是**面向接口编程**。

38

课件截图：接口在体系结构中的作用。接口让层之间依赖抽象，而不是依赖具体实现。

依赖倒置原则（DIP）详解

传统依赖



高层直接依赖低层**具体实现**，耦合度高。

依赖倒置后



高层和低层都依赖**抽象**，依赖方向"倒置"。

DIP 两条规则：

1. 高层模块不应依赖低层模块，两者都应依赖抽象
2. 抽象不应依赖细节，细节应依赖抽象

40

课件截图：依赖倒置原则在体系结构中的应用。上层和下层都围绕接口协作。

2. PO、VO、DAO/Repository

概念	含义	放在哪/用于哪里	考试写法
VO	Value Object，面向展示层的数据传输对象	展示层和逻辑层之间	Service 方法入参/返回值常用 VO，只包含界面需要字段
PO	Persistent Object，持久化对象	逻辑层和数据层之间，或 entity 包	DAO/DataService 方法入参/返回值常用 PO，对应数据库表字段
DAO	Data Access Object，数据访问对象	数据层	封装增删改查，不把 SQL/ORM 细节暴露给逻辑层

概念	含义	放在哪/用于哪里	考试写法
Repository	仓储/数据访问包名，常和 DAO 近似	数据层或领域仓储层	考试中可按 data/repository/dao 命名，但职责要清楚

PO 与 VO 概念

概念	全称	用途	所在层
PO	Persistent Object	与数据库表一一对应	entity / dao
VO	Value Object	面向展示层的数据传输对象	controller / service

为什么需要区分？

- PO 包含数据库细节（主键、外键、时间戳）
- VO 只包含前端需要的字段
- 隔离变化：数据库字段变更不影响前端接口

```
// PO: 对应 book 表
public class Book { Long id; String isbn; String title; BigDecimal price; ... }
// VO: 前端展示用
public class BookVO { String isbn; String title; String priceText; double rating; }
```

39

课件截图：PO 与 VO 概念。记住：VO 给上层看，PO 给持久化用。

接口方法怎么从题目来

展示层到逻辑层接口从用户需求/用例来：用户点什么按钮、输入什么、希望得到什么结果。逻辑层到数据层接口从业务逻辑需要的数据访问来：需要查哪些对象、保存哪些对象、更新哪些状态。

接口设计小结

设计要点	说明
接口 = 契约	定义层间交互的 What, 隐藏 How
PO vs VO	隔离持久化细节与展示需求
DIP	高层和低层都依赖抽象，不直接耦合
构造器注入	Spring 推荐的 DI 方式，保证不可变性
无状态设计	Service 不持有请求级别的状态，确保线程安全

接口设计质量直接决定体系结构的可演化性和可测试性。

45

课件截图：接口设计总结。高质量接口会直接影响可演化性和可测试性。

八、往年卷风格例题与答案

例题 1：进销存系统物理包图

题型来源：2020 风格。系统是 Web 框架实现的进销存系统，按分层体系结构设计，体现所有功能模块、跨网络、浏览器客户端、服务器端逻辑层和数据层、HTTP、REST API。

位置/层	应画出的包或节点
客户端 Browser	html、css、js；sales.view、purchase.view、inventory.view、accounting.view；REST API Client
网络连接	HTTP / REST API，从客户端指向服务器端 Controller 或 Service API
服务器展示/API层	sales.controller、purchase.controller、inventory.controller、accounting.controller
逻辑层	sales.service、purchase.service、inventory.service、accounting.service、schedule.service；对应 Service 接口和实现
数据层	sales.dao、purchase.dao、inventory.dao、accounting.dao、rule.dao；对应 DataService/DAO 接口和实现
对象包	vo 包：SalesOrderVO、PurchaseInquiryVO、InventoryVO；po/entity 包：SalesOrderPO、PurchaseInquiryPO、InventoryPO、ReorderRulePO
数据库	DB，保存产品、库存、销售订单、采购询价单、发票、付款、再订货规则等表

画图时的得分表达

可以用大框表示 Client 和 Server，再在 Server 中画 Presentation/API、Business Logic、Data 三层。箭头从客户端经 HTTP/REST 指向服务器 API，再由 API 调用 Service 接口，Service 调用 DAO 接口，DAO 访问 DB。

例题 2：进销存“运行排程自动生成采购询价单”的接口

问题：针对运行排程自动生成采购询价单，写出展示层和逻辑层接口，以及逻辑层和数据层接口。每个接口写包名。

参考答案思路：展示层接口来自用例动作“运行排程、查看生成结果”；数据层接口来自逻辑层需要查询库存、销售订单、再订货规则，并保存采购询价单。

```

package com.erp.presentation.purchase;

public interface ScheduleAPI {
    ScheduleResultVO runSchedule(String operatorId);
    List<PurchaseInquiryVO> getGeneratedInquiries(String scheduleId);
}

package com.erp.business.purchase;

public interface PurchaseScheduleService {
    ScheduleResultVO runSchedule(String operatorId);
    List<PurchaseInquiryVO> listGeneratedInquiries(String scheduleId);
}

package com.erp.data.inventory;

public interface InventoryDataService {
    List<InventoryPO> findAllInventory();
    InventoryPO findByProductId(String productId);
}

package com.erp.data.sales;

public interface SalesOrderDataService {
    List<SalesOrderPO> findUnfulfilledOrders();
}

package com.erp.data.rule;

public interface ReorderRuleDataService {
    List<ReorderRulePO> findEnabledRules();
    ReorderRulePO findByProductId(String productId);
}

package com.erp.data.purchase;

public interface PurchaseInquiryDataService {
    void save(PurchaseInquiryPO inquiry);
    List<PurchaseInquiryPO> findByScheduleId(String scheduleId);
}

```

- 上层接口使用 VO：ScheduleResultVO、PurchaseInquiryVO。
- 数据层接口使用 PO：InventoryPO、SalesOrderPO、ReorderRulePO、PurchaseInquiryPO。
- 接口按包分布，不要把所有 DAO 都塞进一个巨型接口。
- 不需要写实现类和算法细节，题目要求的是接口声明。

例题 3：校园卡自助系统物理包图

题型来源：2022 风格。系统支持充值、查询余额、绑定银行卡、查询明细等操作；要求分客户端和服务端，体现分层，html/css/js 和展示层在客户端，逻辑层和数据层在服务器端，使用 HTTP 和 REST API。

位置/层	可画内容
客户端 Browser	html、css、js；recharge.view、balance.view、bankcard.view、record.view；REST Client
网络连接	HTTP / REST API
服务器逻辑层	recharge.service、balance.service、bankcard.service、record.service、auth.service
服务器数据层	card.dao、account.dao、bankcard.dao、rechargeRecord.dao、transaction.dao
VO	RechargeRequestVO、RechargeResultVO、RechargeRecordVO、BalanceVO、BankCardVO
PO	CardPO、AccountPO、BankCardPO、RechargeRecordPO、TransactionPO
数据库	校园卡、账户、银行卡绑定、充值记录、交易明细等表

例题 4：校园卡充值服务接口

问题：针对充值服务（充值、查询充值记录）写展示层-逻辑层接口，以及逻辑层-数据层接口。

```
package edu.card.presentation.recharge;

public interface RechargeAPI {
    RechargeResultVO recharge(RechargeRequestVO request);
    List<RechargeRecordVO> queryRechargeRecords(String userId, DateRangeVO range);
}

package edu.card.business.recharge;

public interface RechargeService {
    RechargeResultVO recharge(RechargeRequestVO request);
    List<RechargeRecordVO> queryRechargeRecords(String userId, DateRangeVO range);
}

package edu.card.data.card;

public interface CardDataService {
    CardPO findByCardId(String cardId);
    void updateBalance(String cardId, BigDecimal newBalance);
}

package edu.card.data.bank;

public interface BankCardDataService {
    BankCardPO findBoundBankCard(String userId);
}

package edu.card.data.recharge;

public interface RechargeRecordDataService {
    void save(RechargeRecordPO record);
    List<RechargeRecordPO> findByUserIdAndRange(String userId, DateRangePO range);
}
```

- 展示层只需要知道“充值请求”和“充值结果”，不要返回数据库中的全部账户字段。
- 逻辑层为完成充值，需要查询卡、银行卡绑定、保存充值记录、更新余额，因此拆出对应数据接口。

- 如果题目要求考虑外部支付平台，可补充 required interface：PaymentGateway.pay(...)，但不要把具体支付宝/微信实现写进 Service 接口。

例题 5：概念简答题

题目	答案要点
什么是体系结构三要素？	部件、连接件、配置。部件是功能/数据模块，连接件是交互机制，配置是整体组织和部署拓扑。
分层风格有什么优缺点？	优点：职责清楚、降低耦合、易维护、易替换、易测试。缺点：可能有性能开销，层次过多会增加复杂性。
MVC 中 Model、View、Controller 分别做什么？	Model 表示业务数据和业务状态；View 展示界面；Controller 接收用户输入、分发请求、调用模型或服务。
PO 和 VO 有什么区别？	PO 面向持久化，常对应数据库表；VO 面向展示层数据传输，只包含前端需要字段。
为什么接口设计能降低耦合？	上层依赖抽象接口而不是下层具体实现，具体实现可以替换，测试时可以用桩或模拟对象替代。
包图中依赖方向怎么看？	箭头一般从使用者指向被使用者或其接口。分层结构中高层依赖低层接口，低层不要反向依赖高层。

九、考场速记模板

1. 包图题模板

```
Client / Browser
- html / css / js
- xxx.view / xxx.controller / REST client
    |
    | HTTP + REST API
    v
Server
  Presentation/API Layer
  - xxx.controller or resource
  Business Logic Layer
  - xxx.service interface
  - xxx.service.impl
  - vo package
  Data Layer
  - xxx.dao or repository interface
  - xxx.dao.impl
  - po/entity package
    |
    v
Database
```

2. 接口题模板

- 展示层-逻辑层接口：方法名来自用例动作；参数来自用户输入；返回值来自界面需要展示的结果；使用 VO。
- 逻辑层-数据层接口：方法名来自业务逻辑的数据访问需要；参数通常是 id、条件、PO；返回 PO 或 PO 列表。
- 包名要体现层和模块，例如 `business.recharge`、`data.recharge`、`presentation.recharge`。
- 接口只写声明，不写实现；可以写 2-4 个关键方法，覆盖题目用例即可。

3. 体系结构题万能解释句

- 本系统采用分层体系结构，将界面交互、业务规则和数据访问分离，降低层间耦合。
- 客户端和服务端通过 HTTP/REST API 通信，服务器端隐藏业务实现和数据库细节。
- 逻辑层通过 Service 接口向上提供业务能力，通过 DataService/DAO 接口向下访问数据。
- 展示层与逻辑层之间传递 VO，避免向前端暴露持久化细节；逻辑层与数据层之间传递 PO，便于 ORM 映射和数据库持久化。
- 关键用例可以作为场景视图验证该体系结构，例如从点击按钮到调用 Service、访问 DAO、更新数据库、返回 VO 的完整链路。